K8S SNAPSHOT REPORT

Comprehensive one-week assessment of your Kubernetes costs, security, RBAC, network policies & performance.

PREPARED FOR

SaaSify Tech Inc. (Mock)

Cluster: gke-stg-use1 · Environment: Development

PREPARED BY

Ruben Panussyants

Cloud Architect, CloudNative Inc.
CKA | AWS | Google Cloud | Red Hat Certified

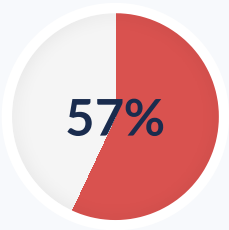
Report Version: v0.4 | Generated: November 12, 2025

Findings valid through December 12, 2025. Re-scan recommended if implementation delayed beyond 30 days.

Executive Dashboard

Your cluster at a glance — the most urgent KPIs surfaced from analysing your Kubernetes environment.

Cluster Health Score



Target: 80%+

Total Savings Potential

$29017.56/year

$2418.13/month unlocked within 30 days

Critical Security Issues

6

CIS FAIL findings requiring immediate action

Reliability Issues

296

Critical: 79 · High: 93 · Medium: 124

RBAC Score

F (0/100)

Cluster-admin SAs: 1 · Wildcards: 19

Target: 80+

Network Policy Coverage

0%

10 pods unprotected

Target: 95%+

> **Risk if unaddressed:** Continuing current trajectory wastes $29017.56/year and leaves 6 critical CIS gaps and 3 high-risk RBAC issues exposed. Poor network segmentation (0% coverage) combined with RBAC gaps creates multiple attack vectors.

Guide:

Executive Summary

Your gke-stg-use1 cluster (Development) scored 57% on our comprehensive health assessment and shows concerning gaps across security, cost, and operational practices. These issues are fixable within two sprints (30 days), and we've identified $29017.56/year in cost savings that will offset remediation investment. This report provides a prioritized roadmap with ready-to-import JIRA tickets.

Summary of Findings

Dimension	Finding	Impact	Timeframe
Cost Efficiency	$29017.56/year waste identified	22% over-provisioned resources draining budget	Fix in Sprint 1-2
Security Posture	6 CIS critical failures	Compliance risk, potential breach vectors	Critical - Fix in Sprint 1
RBAC & Access	Grade F (0/100) - 19 wildcard roles, 3 escalation paths	Privilege escalation paths, security gaps	Critical - Fix in Sprint 1
Network Segmentation	Grade F (0% coverage)	Lateral movement risk, weak isolation	Critical - Fix in Sprint 1
✓ **Performance & Sizing**	9 rightsizing opportunities	Resource contention under load, inefficient scheduling	Optimize in Sprint 2

Sprint 1: Priority Actions

RBAC & Access Controls (CRITICAL)

Issue: 19 roles with wildcard permissions, 3 privilege escalation paths, 1 cluster-admin service accounts detected

Risk: Compromised pod could escalate to cluster-admin, bypassing network controls and gaining full cluster access

Action: Implement least-privilege RBAC, remove wildcard permissions, audit service accounts (3 JIRA tickets ready)

Estimated Time: 2-4 hours

Security Hardening (HIGH)

Issue: 6 critical CIS benchmark failures across: Control Plane Configuration

Risk: Compliance failure, audit findings, increased attack surface for container escapes and control plane compromise

Action: Harden control plane configs, enable Pod Security Standards, fix critical CIS gaps (6 JIRA tickets ready)

Estimated Time: 4-6 hours

Cost Optimization (HIGH VALUE)

Issue: Over-provisioned resources, idle workloads - $29017.56/year annual waste identified

Risk: Budget waste, inefficient resource usage, potential budget exhaustion limiting growth

Action: Apply rightsizing recommendations, remove idle resources, implement autoscaling (13 JIRA tickets ready)

Estimated Time: 12-16 hours

Next Steps & ROI Timeline

We've created **24 import-ready JIRA tickets** prioritized by impact and effort. Sprint 1 focuses on: **RBAC & Access Controls, Security Hardening, Cost Optimization**. Sprint 2 addresses: **Performance optimization**. Estimated total effort: **18-26 hours**. Upon completion, re-scan to verify improvements and achieve 80%+ cluster health score.

ROI Milestones

- **Immediate:** Security risk mitigation (avoid potential $500K+ breach cost)
- **Month 1:** Investment pays for itself after first month of savings ($2418/month × 2 months = $4836)
- **Month 3:** $7254 in cumulative savings (384% ROI)
- **Month 12:** $29018 in cumulative savings (1835% ROI)
- **90+ days:** Cluster health improves from 57% → 80%+

Savings: $29017.56/year | Critical Issues: 6

Cost Savings Analysis

Your cluster's projected monthly spend is **$10736.68**.

⏱️ **Estimated implementation time:** 4-6 hours
💰 **Annual savings potential:** $29017.56/year
📈 **Value:** These optimizations pay for themselves through avoided costs

How We Get to $29017.56/year

Lever	Monthly Impact	Annualized	Next Step	Status
Rightsizing	$117.38/month	$1408.56/year	Apply rightsizing CPU/memory request recommendations for highlighted workloads.	Ready ✓
Idle Cleanup	$2300.75/month	$27609.00/year	Scale down idle namespaces and decommission unused infrastructure (see Idle Resources).	Ready ✓

Note: Monthly wins compound quickly. Implementing both optimizations yields $2418.13/month in recurring savings.

Benchmark vs. Peers

Your cluster's cost efficiency metrics compared to the median for similar SaaS companies:

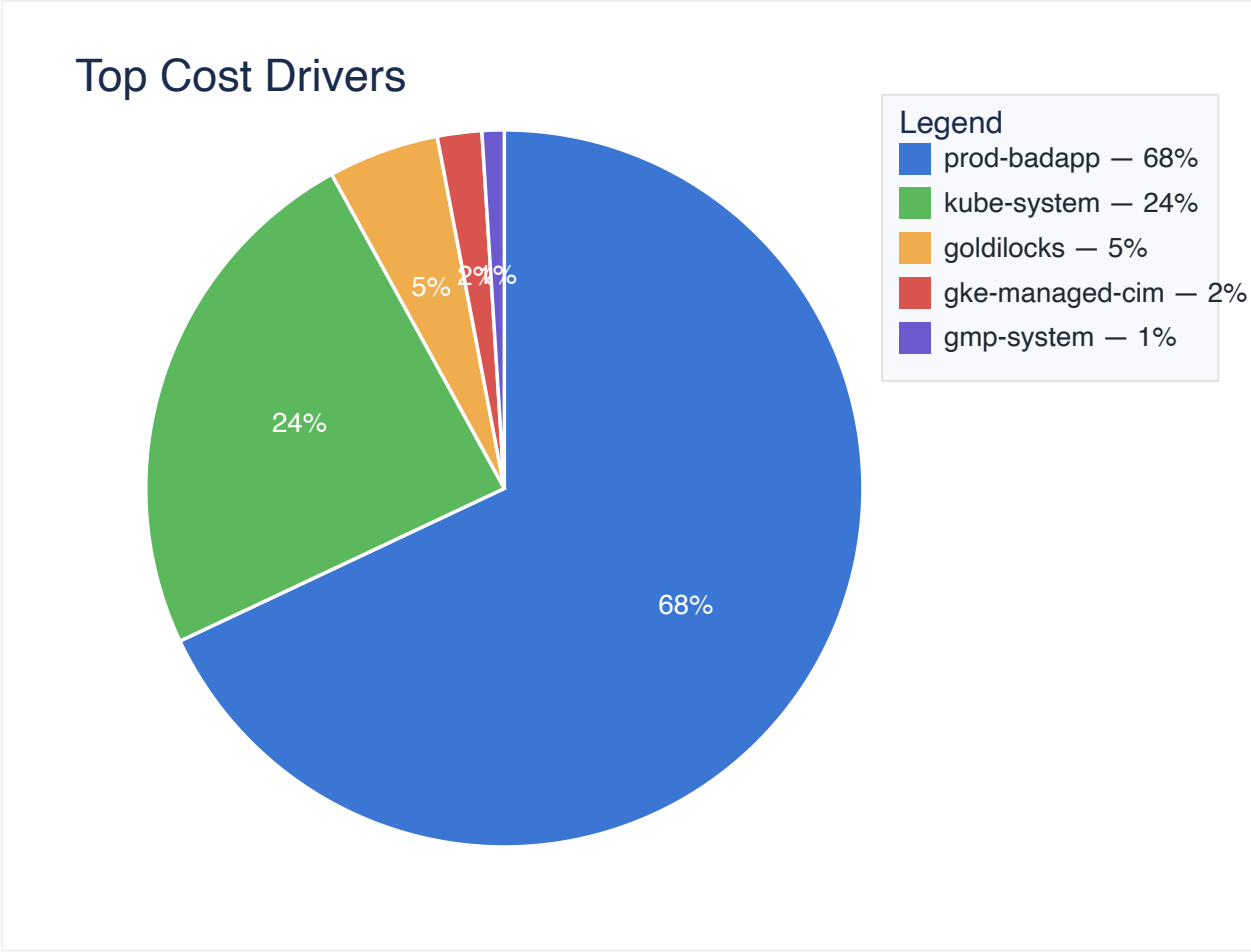
CPU Requests vs Usage

Your Cluster:	**3.1x**
Industry Median:	1.6x
Gap:	+1.5x
Impact:	Paying for unused CPU capacity
Opportunity:	Rightsize requests to actual usage

Namespaces > 2LBs

Your Cluster:	**5**
Industry Median:	1
Gap:	+4
Impact:	Load balancers cost $18-45/month each
Opportunity:	Consolidate or remove unused LBs

Top 5 Cost Drivers by Namespace



Namespace	Est. Monthly Cost	% of Total	Primary Issue
prod-badapp	$7333.50	68%	Over-provisioned resources
kube-system	$2594.87	24%	Over-provisioned resources
goldilocks	$529.11	5%	Over-provisioned resources
gke-managed-cim	$192.90	2%	Over-provisioned resources
gmp-system	$80.36	1%	Right-sized ✓

Note: Remaining $5.94/month spread across other namespaces

Savings: $29017.56/year | Critical Issues: 6

Idle & Wasted Resources

We identified **$2300.75/month** in monthly spend on cloud resources that are providing no value and can be safely decommissioned.

Asset Type	Name / ID	Monthly Cost	Recommendation
Namespace	`prod-badapp`	$1576.80/month	Right-size workloads and decommission idle resources in prod-badapp.
Namespace	`kube-system`	$559.88/month	Right-size workloads and decommission idle resources in kube-system.
Namespace	`goldilocks`	$120.63/month	Right-size workloads and decommission idle resources in goldilocks.
Namespace	`gke-managed-cim`	$43.44/month	Right-size workloads and decommission idle resources in gke-managed-cim.

Each item shows negligible usage during the 7-day window — confirm with service owners, schedule removal in a maintenance window, and monitor workload health post-change.

Rightsizing unlocks $117.38/month each month — details on **Performance, Reliability & Rightsizing**.

Detailed Fact Sheet

A deeper look at the signals behind the dashboard metrics. Use this view to brief engineering and assign ownership.

Category	Status	Key Finding & Recommendation
Cost Efficiency	Warning	**Finding:** Spend ≈ $10736.68 (last 7d). Top driver: prod-badapp (68%). Rightsizing potential: $117.38/month across 9 workload(s) (largest: prod-badapp/badapp, CA $59.96). **Recommendation:** Apply rightsizing recommendations to prod-badapp/badapp to realize ≈ CA $59.96 and validate SLOs after change. Review budgets/owners for prod-badapp and schedule off-hours scaling for non-prod. Scale to zero or remove idle workloads (≈ $2300.75/month).
Security Posture	Critical	**Finding:** CIS score 57% (8/14 passed). Weakest area: Worker Node Security Configuration. **Recommendation:** Triage CIS FAIL items starting with High/High risks; harden node and control plane configs, then re-run kube-bench.
Performance & Stability	Critical	**Finding:** Reliability Grade F (79/93/124 critical/high/medium issues). 79 missing readiness probes, 89 missing resource limits, 4 missing PDBs. 9 rightsizing opportunities (~$117.38/month savings) **Recommendation:** Fix CRITICAL reliability issues immediately (missing probes, limits). Add PodDisruptionBudgets to prevent downtime during maintenance. Apply rightsizing adjustments to high-savings workloads and verify SLOs.
Access Controls	Critical	**Finding:** 66 service accounts, 28 roles, 120 cluster roles. 48 unused accounts; 1 cluster-admin bindings. **Recommendation:** RBAC security score 0/100 (Grade F). Enforce least-privilege, remove unused accounts, and eliminate wildcard permissions.
Network Segmentation	Warning	**Finding:** 0% of pods protected by network policies (0/10 pods), 10 pods without network policies, No default-deny policies found, and 1 high-severity findings. **Recommendation:** Implement network policies to increase pod coverage to 80%+, Apply default-deny policies to critical namespaces, and Add egress controls to restrict outbound traffic.

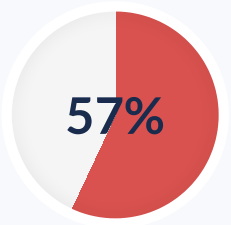
Savings: $29017.56/year | Critical Issues: 6

Security & Hardening Audit

We audited the cluster against the industry-standard Center for Internet Security (CIS) Benchmark. The CIS Benchmark is a globally recognized set of best practices for securing technology systems, providing a vendor-neutral, objective measure of your security posture.

> **Note on Managed Kubernetes:** This cluster runs on GKE (managed Kubernetes). Some CIS findings relate to worker node configurations managed by Google Cloud Platform. We've highlighted findings where customer action is possible or where Google's defaults should be reviewed against your security requirements. Control plane checks are managed by Google and not included in this assessment.

CIS Score



Industry benchmark: 80%+

Critical FAIL Items

6

6 total checks failed

Remediate the next page before promoting this cluster to production.

Risk Heatmap

This heatmap visualizes the distribution of findings based on impact and likelihood. The numbers represent the count of failed checks in each category.

Impact / Likelihood	Low	Medium	High
High Impact	0	0	0
Medium Impact	0	0	6
Low Impact	0	0	0

CIS Score: 57% (8/14 checks passed; 6 outstanding).

> ⚠️ **Action Required:** 6 critical CIS checks require immediate attention. See **Top Critical Security Findings** section (pages later in this report) for detailed remediation steps, JIRA tickets, and acceptance criteria for each failing check.

CIS Domain	Passed	Total
Worker Node Security Configuration	8	14
Kubernetes Policies	0	0
Managed Services	0	0

Savings: $29017.56/year | Critical Issues: 6

RBAC & Access Controls

Access reviews highlight opportunities to reduce blast radius and align with least-privilege principles.

> **RBAC Security Score:** 0/100 (Grade F). We subtracted points for cluster-admin bindings, wildcard permissions, and unused service accounts.

Metric	Count
Total ServiceAccounts	66
Total Roles	28
Total ClusterRoles	120
Unused ServiceAccounts	48
ServiceAccounts with cluster-admin	1
Roles with wildcard permissions	19

Critical Findings

1 ServiceAccounts with cluster-admin (HIGH)

These service accounts are bound to cluster-admin and have unrestricted access across the cluster.

Affected:

- prod-badapp/badapp-sa

Recommendation: Review and replace cluster-admin bindings with scoped roles or remove access entirely.

Ticket: SEC-RBAC-CLUSTER-ADMIN

19 Roles with wildcard permissions (HIGH)

Roles that grant * verbs or * resources provide wide attack surface and complicate auditing.

Affected:

- badapp-cluster-role
- cluster-admin
- external-metrics-reader
- kubelet-api-admin
- system:cloud-controller-manager
- system:controller:generic-garbage-collector
- system:controller:horizontal-pod-autoscaler
- system:controller:namespace-controller
- system:controller:resourcequota-controller
- system:gcp-controller-manager
- system:gke-common-webhooks
- system:gke-hpa-actor
- … +7 more

Recommendation: Replace wildcard permissions with explicit verb/resource combinations aligned with least privilege.

Ticket: SEC-RBAC-WILDCARD

48 unused service accounts detected (MEDIUM)

Unused service accounts retain credentials that can be abused by attackers.

Affected:

- default/default
- gke-managed-cim/default
- gke-managed-system/default
- gke-managed-volumepopulator/default
- gmp-public/default
- gmp-system/default
- kube-node-lease/default
- kube-public/default
- kube-system/attachdetach-controller
- kube-system/certificate-controller
- kube-system/cloud-provider
- kube-system/clusterrole-aggregation-controller
- … +36 more

Recommendation: Delete unused accounts or disable automountServiceAccountToken to avoid storing unnecessary credentials.

Ticket: SEC-RBAC-UNUSED-SA

Privilege Escalation Paths

From	To	Path	Risk
admin	impersonation	ClusterRole admin can impersonate users/groups (verbs: impersonate, resources: serviceaccounts)	HIGH
edit	impersonation	ClusterRole edit can impersonate users/groups (verbs: impersonate, resources: serviceaccounts)	HIGH
system:aggregate-to-edit	impersonation	ClusterRole system:aggregate-to-edit can impersonate users/groups (verbs: impersonate, resources: serviceaccounts)	HIGH

Unused ServiceAccounts

Detected tokens without workload usage over the inspected period. Remove or disable to reduce attack surface.

- default/default
- gke-managed-cim/default
- gke-managed-system/default
- gke-managed-volumepopulator/default
- gmp-public/default
- gmp-system/default
- kube-node-lease/default
- kube-public/default
- kube-system/attachdetach-controller
- kube-system/certificate-controller
- kube-system/cloud-provider
- kube-system/clusterrole-aggregation-controller
- kube-system/cronjob-controller
- kube-system/daemon-set-controller
- kube-system/deployment-controller
- kube-system/disruption-controller
- kube-system/efficiency-daemon
- kube-system/endpoint-controller
- kube-system/endpointslice-controller
- kube-system/endpointslicemirroring-controller
- kube-system/ephemeral-volume-controller
- kube-system/expand-controller
- kube-system/generic-garbage-collector
- kube-system/gke-system-balloon-pod
- kube-system/gpu-device-plugin-sa
- … +23 more

Savings: $29017.56/year | Critical Issues: 6

Network Policy Analysis

Network policies provide critical microsegmentation to prevent lateral movement and limit blast radius in case of pod compromise.

> **Network Policy Security Score:** 0/100 (Grade F). Policy Coverage: 0% · Isolated Namespaces: 0/2

Summary Metrics

Metric	Value	Status
Total Namespaces	2	-
Namespaces with Policies	0	⚠️ LOW
Total Pods	10	-
Pods Protected by Policies	0 (0%)	❌ CRITICAL
Network Policies Defined	0	⚠️ LOW
Default-Deny Policies	0	❌ NONE
Pods with Unrestricted Egress	10	⚠️

Policy Coverage by Namespace

Namespace	Total Pods	Protected Pods	Coverage
prod-badapp	10	0	0% ⚠️

Critical Findings

[HIGH] Database Pods Accessible From All Namespaces

Affected:

- prod-badapp/badapp-db-0
- prod-badapp/badapp-db-1
- prod-badapp/badapp-idle-fdbc949b5-hpgpj
- prod-badapp/badapp-idle-fdbc949b5-pljt4
- prod-badapp/kubecost-grafana-5688b4fdb5-6cfm6

Risk: Any compromised pod in ANY namespace can access databases. No segmentation between environments.

Recommendation: Restrict database access to only application pods that require it using podSelector

JIRA Ticket: SEC-NET-002

[MEDIUM] No Egress Policies

Affected:

- 10 pods (100% of cluster workloads)

Risk: Pods can communicate with any external service. Data exfiltration is trivially easy. No visibility into required external dependencies.

Recommendation: Implement egress policies per namespace, whitelist required external services (DNS, cloud APIs, etc.)

JIRA Ticket: SEC-NET-003

Current vs. Target State

Metric	Current	Target	Status
Policy Coverage	0%	95%+	⚠ BELOW TARGET
Namespaces with Policies	0/2	10/10	⚠ BELOW TARGET
Default-Deny Policies	0	10	⚠ BELOW TARGET
Network Policy Score	0/100	85+	⚠ BELOW TARGET

Network Segmentation Score: 0/100 (F)

Top Critical Security Findings

The following `[FAIL]` results from the CIS scan represent critical risks and should be remediated immediately.

3.2.1: Ensure that the --anonymous-auth argument is set to false (Automated)

Risk: Medium impact, High likelihood; cluster remains non-compliant with CIS.

Recommendation: `Follow CIS 3.2.1 to harden worker node security: Ensure that the --anonymous-auth argument is set to false (Automated).`

3.2.2: Ensure that the --authorization-mode argument is not set to AlwaysAllow (Automated)

Risk: Medium impact, High likelihood; cluster remains non-compliant with CIS.

Recommendation: `Follow CIS 3.2.2 to harden worker node security: Ensure that the --authorization-mode argument is not set to AlwaysAllow (Automated).`

3.2.3: Ensure that the --client-ca-file argument is set as appropriate (Automated)

Risk: Medium impact, High likelihood; cluster remains non-compliant with CIS.

Recommendation: `Follow CIS 3.2.3 to harden worker node security: Ensure that the --client-ca-file argument is set as appropriate (Automated).`

3.2.6: Ensure that the --protect-kernel-defaults argument is set to true (Manual)

Risk: Medium impact, High likelihood; cluster remains non-compliant with CIS.

Recommendation: `Follow CIS 3.2.6 to harden worker node security: Ensure that the --protect-kernel-defaults argument is set to true (Manual).`

3.2.9: Ensure that the --event-qps argument is set to 0 or a level which ensures appropriate event capture (Automated)

Risk: Medium impact, High likelihood; cluster remains non-compliant with CIS.

Recommendation: `Follow CIS 3.2.9 to harden worker node security: Ensure that the --event-qps argument is set to 0 or a level which ensures appropriate event capture (Automated).`

3.2.12: Ensure that the RotateKubeletServerCertificate argument is set to true (Automated)

Risk: Medium impact, High likelihood; cluster remains non-compliant with CIS.

Recommendation: `Follow CIS 3.2.12 to harden worker node security: Ensure that the RotateKubeletServerCertificate argument is set to true (Automated).`

Savings: $29017.56/year | Critical Issues: 6

Performance, Reliability & Rightsizing Analysis

> **Note:** This assessment includes both user workloads and system components for completeness. In managed Kubernetes environments like GKE, some system component configurations are managed by the provider. We prioritize findings by actionability during remediation planning.

Workload Reliability Assessment

Analyzed **46 workloads** for production readiness. Reliability Grade: F (0/100)

Severity	Workload	Type	Issue	Impact	Effort
CRITICAL	`gmp-system/rule-evaluator`	Deployment	Container 'config-reloader' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/event-exporter-gke`	Deployment	Container 'event-exporter' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/event-exporter-gke`	Deployment	Container 'prometheus-to-sd-exporter' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/konnectivity-agent`	Deployment	Container 'konnectivity-agent' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/konnectivity-agent`	Deployment	Container 'konnectivity-agent-metrics-collector' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/konnectivity-agent-autoscaler`	Deployment	Container 'autoscaler' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/kube-dns`	Deployment	Container 'dnsmasq' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/kube-dns`	Deployment	Container 'sidecar' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/kube-dns`	Deployment	Container 'kubedns-metrics-collector' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during rollouts and restarts	1 hour
CRITICAL	`kube-system/kube-dns-autoscaler`	Deployment	Container 'autoscaler' missing readiness probe	Traffic routed to unhealthy pods → 500 errors during	1 hour

Severity	Workload	Type	Issue	Impact	Effort
				rollouts and restarts	
HIGH	`gmp-system/gmp-operator`	Deployment	Container 'operator' missing CPU limit	Pod can starve other workloads on node → cascading performance degradation	30 minutes
HIGH	`gmp-system/rule-evaluator`	Deployment	Container 'config-reloader' missing CPU limit	Pod can starve other workloads on node → cascading performance degradation	30 minutes
HIGH	`gmp-system/rule-evaluator`	Deployment	Container 'evaluator' missing CPU limit	Pod can starve other workloads on node → cascading performance degradation	30 minutes
HIGH	`kube-system/event-exporter-gke`	Deployment	Container 'event-exporter' missing CPU limit	Pod can starve other workloads on node → cascading performance degradation	30 minutes
HIGH	`kube-system/event-exporter-gke`	Deployment	Container 'event-exporter' missing memory limit	Pod can cause OOMKill cascade → node instability and service disruption	30 minutes
HIGH	`kube-system/event-exporter-gke`	Deployment	Container 'prometheus-to-sd-exporter' missing CPU limit	Pod can starve other workloads on node → cascading performance degradation	30 minutes
HIGH	`kube-system/event-exporter-gke`	Deployment	Container 'prometheus-to-sd-exporter' missing memory limit	Pod can cause OOMKill cascade → node instability and service disruption	30 minutes
HIGH	`kube-system/konnectivity-agent`	Deployment	Container 'konnectivity-agent' missing CPU limit	Pod can starve other workloads on node → cascading performance degradation	30 minutes

Severity	Workload	Type	Issue	Impact	Effort
HIGH	`kube-system/konnectivity-agent`	Deployment	Missing PodDisruptionBudget	Voluntary disruptions (node drain, upgrade) can take down all replicas → downtime	30 minutes
HIGH	`kube-system/konnectivity-agent-autoscaler`	Deployment	Container 'autoscaler' missing CPU limit	Pod can starve other workloads on node → cascading performance degradation	30 minutes
MEDIUM	`gmp-system/rule-evaluator`	Deployment	Container 'config-reloader' missing liveness probe	Stuck pods won't auto-restart → prolonged service degradation	1 hour
MEDIUM	`kube-system/event-exporter-gke`	Deployment	Container 'event-exporter' missing liveness probe	Stuck pods won't auto-restart → prolonged service degradation	1 hour
MEDIUM	`kube-system/event-exporter-gke`	Deployment	Container 'prometheus-to-sd-exporter' missing liveness probe	Stuck pods won't auto-restart → prolonged service degradation	1 hour
MEDIUM	`kube-system/event-exporter-gke`	Deployment	Single replica in production	No high availability → downtime during deployments and node failures	1 hour
MEDIUM	`kube-system/konnectivity-agent`	Deployment	Container 'konnectivity-agent-metrics-collector' missing liveness probe	Stuck pods won't auto-restart → prolonged service degradation	1 hour
MEDIUM	`kube-system/konnectivity-agent`	Deployment	Missing HorizontalPodAutoscaler	Cannot handle traffic spikes → service degradation during peak load	2 hours
MEDIUM	`kube-system/konnectivity-agent-autoscaler`	Deployment	Container 'autoscaler' missing liveness probe	Stuck pods won't auto-restart → prolonged service degradation	1 hour

Severity	Workload	Type	Issue	Impact	Effort
MEDIUM	`kube-system/konnectivity-agent-autoscaler`	Deployment	Single replica in production	No high availability → downtime during deployments and node failures	1 hour
MEDIUM	`kube-system/kube-dns`	Deployment	Container 'dnsmasq' may run as root	Privilege escalation risk if container is compromised	2 hours
MEDIUM	`kube-system/kube-dns`	Deployment	Container 'kubedns-metrics-collector' missing liveness probe	Stuck pods won't auto-restart → prolonged service degradation	1 hour

Resource Rightsizing Opportunities

Many workloads have requested significantly more CPU and Memory than they consume, leading to resource waste and inefficient scheduling. Beyond just cost, this can lead to 'bin packing' problems where nodes are unable to schedule new pods, even with available capacity, causing deployment delays and potential instability.

Workload	Current Request (CPU/Mem)	Recommended Request (CPU/Mem)	Est. Monthly Savings
prod-badapp/badapp	2000m CPU / 4096Mi Mem	**50m CPU / 128Mi Mem**	**CA $59.96**
prod-badapp/badapp-db	1000m CPU / 2048Mi Mem	**50m CPU / 128Mi Mem**	**CA $29.17**
prod-badapp/badapp-idle	500m CPU / 1024Mi Mem	**50m CPU / 128Mi Mem**	**CA $13.77**
kube-system/kube-dns	270m CPU / 155Mi Mem	**50m CPU / 128Mi Mem**	**CA $5.43**
prod-badapp/kubecost-forecasting	200m CPU / 300Mi Mem	**50m CPU / 186Mi Mem**	**CA $4.00**
kube-system/fluentbit-gke	105m CPU / 230Mi Mem	**50m CPU / 128Mi Mem**	**CA $1.66**
gke-managed-cim/kube-state-metrics	105m CPU / 130Mi Mem	**50m CPU / 128Mi Mem**	**CA $1.34**
prod-badapp/kubecost-cost-analyzer	210m CPU / 110Mi Mem	**57m CPU / 887Mi Mem**	**CA $1.24**
kube-system/kube-proxy	100m CPU / 0Mi Mem	**50m CPU / 128Mi Mem**	**CA $0.81**

Note: Recommendations based on 7-day usage data from Development environment. Actual production requirements may differ based on traffic patterns and load testing. Minimum recommendations enforced: 50m CPU / 128Mi memory for stability under load. Validate in staging before applying to production.

Savings: $29017.56/year | Critical Issues: 6

Prioritized Action Plan

This roadmap summarizes our findings into a prioritized action plan. We have prioritized these items based on a balance of impact—considering cost, security risk, and performance—and the estimated effort required for remediation, allowing your team to focus on the quickest, most impactful wins first.

Ticket	Priority	Finding	Recommended Action	Impact	Effort
SEC-NET-002	**High**	Database Pods Accessible From All Namespaces	Restrict database access to only application pods that require it using podSelector	Any compromised pod in ANY namespace can access databases. No segmentation between environments.	Medium
Acceptance: Network policy applied and validated for: prod-badapp/badapp-db-0 and 4 more					
SEC-NET-003	**Medium**	No Egress Policies	Implement egress policies per namespace, whitelist required external services (DNS, cloud APIs, etc.)	Pods can communicate with any external service. Data exfiltration is trivially easy. No visibility into required external dependencies.	Medium
Acceptance: Network policy applied and validated for: 10 pods (100% of cluster workloads)					
SEC-RBAC-CLUSTER-ADMIN	**P1**	RBAC: 1 ServiceAccounts with cluster-admin	Review and replace cluster-admin bindings with scoped roles or remove access entirely.	Addresses RBAC gap: 1 ServiceAccounts with cluster-admin	1 day
Acceptance: RBAC bindings updated, validated via kubectl auth can-i and regression tests.					
SEC-RBAC-WILDCARD	**P1**	RBAC: 19 Roles with wildcard permissions	Replace wildcard permissions with explicit verb/resource combinations aligned with least privilege.	Addresses RBAC gap: 19 Roles with wildcard permissions	1 day
Acceptance: RBAC bindings updated, validated via kubectl auth can-i and regression tests.					
SEC-RBAC-UNUSED-SA	**P2**	RBAC: 48 unused service accounts detected	Delete unused accounts or disable automountServiceAccountToken to avoid storing unnecessary credentials.	Addresses RBAC gap: 48 unused service accounts detected	0.5 day
Acceptance: RBAC bindings updated, validated via kubectl auth can-i and regression tests.					
P1-OPS-001	**P1**	Anonymous auth enabled on kubelet	Set --anonymous-auth=false and restart kubelet	Blocks unauthenticated kubelet access	0.5 day
Acceptance: Changes applied, validated via rerun of relevant scan, and monitoring confirms improvement.					

Ticket	Priority	Finding	Recommended Action	Impact	Effort
P1-OPS-002	**P1**	Over-provisioned productcatalogservice	Apply rightsizing recommendation 150m/128Mi	Saves ~$90/month and stabilizes autoscaling	0.5 day
Acceptance: Changes applied, validated via rerun of relevant scan, and monitoring confirms improvement.					

Ready-to-import Jira ticket bundle: `out/jira_action_plan.csv`

💰 **Savings: $29017.56/year** | 🛡️ **Critical Issues: 6**

Next Steps

Acting on these findings is the most critical step in improving your cluster's health. To support your team, we offer structured engagement models designed to accelerate remediation and lock in the benefits identified in this report.

Option 1: Fix & Verify Sprint

A focused engagement to remediate all Critical and High-priority findings from this report. We co-deliver fixes with your team, re-run the scans to verify results, and provide the before/after evidence pack for stakeholders.

- **Timeline:** 3-4 weeks
- **Investment:** CA $4,950 (US $3,700)
- **ROI:** Total investment of CA $11,900 (assessment CA $6,950 + implementation CA $4,950 (US $3,700)) pays for itself in 5 months from identified savings, delivering 240%+ ROI in year one.

Option 2: Self-Service (DIY)

Use this report as your implementation roadmap. Each finding includes detailed remediation steps, acceptance criteria, and effort estimates. We remain available for ad-hoc consultation at our standard hourly rate if your team encounters blockers.

- **Timeline:** At your own pace
- **Investment:** Assessment only (CA $6,950)
- **Support:** Ad-hoc consultation available at CA $200/hour

Methodology & Data Privacy

Methodology

This report is generated through a combination of automated scanning and expert analysis. We use a suite of industry-standard, open-source tools to gather objective data across three key pillars:

- **Cost Efficiency:** Assessed using Kubecost to analyze cloud spend allocation and identify idle or wasted resources.
- **Security Posture:** Audited using Kube-bench to check cluster configurations against the CIS (Center for Internet Security) Benchmark.
- **Performance & Rightsizing:** Analyzed using Metrics Server to compare actual resource usage against workload requests.

Kubernetes Resource Collection Process

To analyze your cluster's RBAC policies, network policies, and workload configurations, we capture Kubernetes resource definitions in JSON format using standard `kubectl` commands. This process:

- **Captures Configuration Data Only:** We export resource manifests (RBAC roles, network policies, deployments, pods) using `kubectl get -o json`. This includes metadata and configuration—not application data or secrets.
- **No Application Data:** We do not access environment variables containing secrets, mounted volumes, or application logs. Our tooling filters out sensitive fields automatically.
- **Secure Analysis:** Resource JSON files are transmitted securely and analyzed by our custom-built analysis engine. We identify misconfigurations, overprivileged roles, and missing security controls, then purge all raw data after report generation.

Our team synthesizes this data, prioritizes findings based on business impact, and provides actionable, context-aware recommendations.

Data Privacy & Security

We are committed to the highest standards of data privacy and security throughout the engagement.

- **Guided Install Model:** Your team deploys our diagnostic tools (Kubecost, Kube-bench, Metrics Server) within your cluster. We never receive direct credentials or cluster access.
- **Data Handling:** Your team exports the JSON output from these tools and shares it with us via secure channels. We analyze this configuration data to generate the report, then purge all raw files upon completion.
- **No Secrets Exposure:** The exported data contains resource configurations and metadata only—no application secrets, environment variables, or sensitive runtime data.
- **Tool Cleanup:** All diagnostic tools and their configurations are completely removed from your cluster at the end of the engagement, leaving your environment in its original state.

Glossary

CIS Benchmark
: A set of globally recognized best practices for securing technology systems, developed by the Center for Internet Security.

Namespace
: A logical grouping of objects within a Kubernetes cluster, used to scope and isolate resources for different teams or applications (e.g., `prod`, `dev`).

Pod
: The smallest deployable unit in Kubernetes, representing a single instance of a running process in a cluster. It contains one or more containers.

CPU/Memory Request
: The amount of CPU and Memory that a container is guaranteed to get. Kubernetes uses this value for scheduling pods onto nodes.

Rightsizing
: The process of adjusting resource requests to match a workload's actual usage, eliminating waste and improving efficiency.

Idle Resource
: A cloud asset (like a disk or load balancer) that is still provisioned and incurring costs but is not actively being used by any application.